

AUDITORIA DE SEGURANÇA · CÓDIGO E BANCO DE PRODUÇÃO

Relatório de Auditoria de Segurança: UseMarcon

Data: **07/10/2026** · Revisão: ef1c099 (branch claude/epic-hamilton-638849, base local-work-marcon)

Categorias: banco sem tranca, permissão no navegador, IDOR, chaves expostas e XSS

Escopo auditado

- Código da aplicação em src/ (Next.js 16, App Router): 17 arquivos de server actions (60 ações exportadas), 6 route handlers, páginas públicas /loja e /r.
- 29 migrations em supabase/migrations e o estado real do banco de produção (projeto ouvxsfefegqcvwmdume), lido só com consultas de leitura: políticas RLS, views, funções SECURITY DEFINER, gatilhos, FKs, CHECKs, privilégios e Storage.
- Scripts em scripts/, README.md, vercel.json, next.config.ts, .gitignore e o histórico git completo (155 commits).
- Fora do escopo: painel da Vercel, painel do Asaas, configuração de Auth no painel do Supabase e testes dinâmicos contra produção (nada foi escrito no banco).

Resultado em uma linha

11 achados verificados: 2 altas, 2 médias, 2 baixas, 5 informativas. Nenhum crítico. 16 controles conferidos e corretos. Severidade máxima: **Alta**

Toda afirmação deste relatório foi conferida no código ou por consulta somente leitura ao banco de produção. Nada foi alterado no banco durante a auditoria.

Stack detectada e nota metodológica

Linguagem	TypeScript (Node 24), SQL/PLpgSQL
Framework	Next.js 16.3 com App Router, Server Actions e proxy.ts (antigo middleware)
Banco e acesso	Supabase Postgres via @supabase/supabase-js e @supabase/ssr (PostgREST). Sem ORM.
Autenticação	Supabase Auth (e-mail e Google), sessão em cookie; admin por lista ADMIN_EMAILS no servidor
Isolamento	RLS por owner_id = auth.uid() em todas as tabelas; funções RPC com filtro manual; cliente service role em pontos específicos
Armazenamento	Supabase Storage, 3 buckets privados com política por pasta do usuário
Frontend	React 19 + Tailwind 4, PWA com push (web-push)
Deploy	Vercel (vercel.json com cron). Sem Docker, Helm, Terraform ou CI no repositório
Integrações	Asaas (cobrança, webhook), Anthropic (IA de anúncios), Google Analytics, Microsoft Clarity

Como cada categoria foi mapeada para esta stack

1. Banco sem tranca	Mecanismo do projeto: RLS do Postgres com owner_id = auth.uid(). Conferido em produção: RLS ligado e políticas em cada tabela, views com security_invoker, funções SECURITY DEFINER (que ignoram RLS) e os pontos onde o servidor usa a chave de serviço. Também as FKs, porque a checagem de FK ignora RLS.
2. Permissão definida no navegador	Cruzamento de cada gate de interface (menu Admin, recursos do plano pago, cota de IA) com a checagem no servidor ou no banco. Inclui limites de plano cuja contagem depende de dados que o próprio usuário pode alterar pela API REST, o que equivale a confiar no cliente.
3. IDOR	Todas as server actions exportadas (60 funções em 17 arquivos 'use server') e os 6 route handlers, um por um: de onde vem cada ID, qual cliente Supabase é usado (sessão com RLS ou service role) e se a posse do objeto e dos objetos relacionados é conferida.
4. Chaves expostas	Busca por padrões de chave (Anthropic, Supabase JWT, Asaas, AWS, chaves privadas, senhas, defaults \${VAR:-x}) no código, scripts, docs e no histórico git inteiro; variáveis NEXT_PUBLIC_ que entram no bundle; componentes 'use client' que importem módulos com segredo.
5. Inputs sem tratamento (XSS)	Busca por dangerouslySetInnerHTML, innerHTML, eval, new Function, srcdoc, window.open e href/src montados com dado do usuário; renderização de markdown/HTML (não há lib); dados do lojista exibidos na loja e no recibo públicos; CHECKs do banco que repetem a validação do formulário.

Escala de severidade

Crítica	Exploração remota simples que expõe ou altera dados de outras contas em massa, ou dá controle da plataforma.
Alta	Exploração simples com impacto real em outra conta ou custo direto para o negócio.
Média	Impacto real, mas com pré-condição que limita (conhecer um dado difícil de obter, papel específico).
Baixa	Impacto pequeno ou exige posição privilegiada (rede, máquina do desenvolvedor).
Informativa	Sem exploração conhecida; endurecimento e defesa em profundidade.

Resumo executivo



Leitura

O isolamento por dono está bem montado onde o projeto pensou nele: RLS em todas as tabelas, views com `security_invoker`, buckets privados e admin decidido no servidor. Os dois riscos altos ficam nas bordas desse desenho. Primeiro, o RLS protege cada linha, mas não a relação entre linhas, e a loja pública confia nessa relação (A1). Segundo, limites que custam dinheiro são contados em tabelas que o próprio usuário pode apagar (A3). Nenhum segredo foi encontrado no repositório nem no histórico, e não há ponto de XSS.

Categoria	Achados	Mais grave	Pontos fortes
1. Banco sem tranca	5	Alta	5
2. Permissão definida no navegador	1	Alta	2
3. IDOR	1	Média	4
4. Chaves expostas	3	Baixa	2
5. Inputs sem tratamento (XSS)	1	Informativa	3

Pontos fortes e pontos fracos

Pontos fracos (riscos centrais)

- O RLS protege a linha, mas não a relação: nenhuma política nem FK garante que o `produto_id` de uma linha filha é do mesmo dono, e funções SECURITY DEFINER públicas confiam nessa relação (A1, A2).
- Limites que custam dinheiro (IA) e que sustentam o plano pago (vendas/mês) são contados em tabelas que o próprio usuário apaga e edita (A3).
- A service role assina caminhos de arquivo que vêm de colunas e metadados editáveis pelo usuário, sem conferir a pasta do dono (A4). Validações feitas só na server action são contornadas pela API REST.
- Endpoints públicos sem limite de taxa (métricas e cupons) e pequenos endurecimentos pendentes (A5 a A11).

Pontos fortes (verificados, com evidência)

- **Isolamento** RLS ligado em 100% das 30 tabelas do schema public, com políticas `owner_id = auth.uid()` e WITH CHECK nos UPDATES (conferido em produção, não só nas migrations).
- **Isolamento** As duas views (`produtos_com_estoque`, `parcelas_com_status`) têm `security_invoker=true`: respeitam o RLS de quem consulta.
- **Isolamento** Buckets logo-empresa, produto-fotos e vitrine-banner privados, com política por pasta (`storage.foldername(name))[1] = auth.uid()`).
- **Isolamento** Funções públicas (`vitrine_publica`, `vitrine_banner`, `validar_cupom_vitrine`) só respondem para loja ativa e plano pago/teste e não expõem custo, estoque exato nem dados de cliente; `recibo_publico` exige token UUID aleatório (`gen_random_uuid`) e devolve só campos do recibo.
- **Isolamento** Tabelas financeiras sensíveis (`assinaturas`, `comissoes`, `saques`, `indicacoes`, `asaas_eventos`) só têm SELECT do dono ou nenhuma política: escrita só pela service role ou por função `definer` (`pedir_saque` confere uid e saldo com lock).
- **Permissão** Admin decidido no servidor por `ADMIN_EMAILS` + e-mail confirmado, nunca por `user_metadata` (`src/lib/admin.ts:13-17`). O menu esconde o link (`layout.tsx:24`) e o backend repete a checagem em `/admin` (`page.tsx:49`), `/admin/indicacoes` (`page.tsx:28`) e em todas as ações (`admin/actions.ts:18 e 97`).
- **Permissão** Recursos do plano pago (`importar histórico`, `fotos extras`, `vitrine`, `banner`, `cupons`) barrados no banco por gatilho ou na função pública, não só na interface.
- **IDOR** Todas as server actions usam o cliente da sessão: `update/delete` por id passam pelo RLS (clientes, fornecedores, lançamentos, cupons, variações, fotos, anúncios, canais, formas de pagamento).
- **IDOR** As RPCs de escrita (`registrar_venda`, `cancelar_venda`, `pagar_parcela`, `estoque`, `excluir_produto`, `novo_link_recibo`, `salvar_anuncio_versao`) filtram `owner_id = auth.uid()` em toda leitura e escrita.
- **IDOR** Usos da service role têm escopo fixo: assinatura sempre `.eq('owner_id', user.id)`; `/indique` só consulta ids que o RLS já devolveu; `webhook` e `cron` não recebem ids de usuário.

- **IDOR** Webhook do Asaas exige token com timingSafeEqual e é idempotente; cron fechado sem CRON_SECRET; callback de auth só redireciona para caminho interno (bloqueia // e prefixa a origem).
- **Segredos** Nenhum segredo no código, scripts, README ou nos 155 commits do histórico; .env* no .gitignore; scripts leem a senha do .env.local.
- **Segredos** Só vão para o bundle variáveis públicas por natureza (URL e anon key do Supabase, chave pública VAPID, ids de GA e Clarity). Nenhum componente 'use client' importa módulo com segredo. Sem defaults \${VAR:-x} nem arquivos de Docker/CI.
- **XSS** Único dangerouslySetInnerHTML é um script estático de tema (layout.tsx:37-41). Ids de GA e Clarity validados por regex antes de entrar no script (analytics.tsx:8-9).
- **XSS** Sem renderizador de markdown/HTML; todo texto do lojista e do cliente passa pelo escape do React; o PDF do recibo usa texto do pdf-lib.
- **XSS** Links com dado do usuário têm prefixo fixo (https://wa.me/, https://instagram.com/) e o banco repete a validação com CHECK (whatsapp só dígitos, instagram [A-Za-z0-9._], cor #RRGGBB, slug).

Achados por categoria

1. Banco sem tranca

Id	Severidade	Arquivo:linha	Descrição
A1	Alta	supabase/migrations/0001_fase1_produtos_es toque_caixa.sql:96-97 supabase/migrations/0001_fase1_produtos_es toque_caixa.sql:122-123 supabase/migrations/0026_vitrine_extras.sq l:71 e 75 (vitrine_publica, SECURITY DEFINER, em produção) supabase/migrations/0011_limite_de_fotos.s ql:6-14 (limitar_fotos_produto)	Tabelas filhas aceitam produto de outro dono e a loja pública mostra essas linhas
A4	Média	src/lib/vitrine-servidor.ts:26-28, 31-32 e 36-38 src/app/r/[token]/page.tsx:33-36 e src/app/r/[token]/pdf/route.ts:47 src/app/(app)/configuracoes/actions.ts:58- 66 src/app/(app)/vitrine/actions.ts:113 (validação só na action)	A chave de serviço assina arquivos do Storage cujo caminho o usuário escolhe
A5	Baixa	supabase/migrations/0028_vitrine_cupons_me tricas.sql:87-117 src/app/loja/[slug]/actions.ts:27-35	Métricas da vitrine podem ser infladas por qualquer pessoa, sem limite
A7	Informativa	produção: proacl de ajustar_estoque, cancelar_venda, excluir_produto, pagar_parcela, registrar_entrada/saida_estoque, registrar_venda, uso_do_plano, aplicar_limites_* supabase/migrations/0029_revogar_execute_a non.sql	13 funções SECURITY DEFINER executáveis por anon (correção já em PR)
A11	Informativa	supabase/migrations/0028_vitrine_cupons_me tricas.sql:39 e 56-57	Validação de cupom sem limite de tentativas

2. Permissão definida no navegador

Id	Severidade	Arquivo:linha	Descrição
A3	Alta	supabase/migrations/0017_ia_anuncios_e_rec ibo.sql:29-33 função ia_do_mes (produção), usada pelo gatilho aplicar_limites_ia supabase/migrations/0003_clientes_vendas_p arcelas.sql:63-64 + vendas_do_mes src/app/(app)/anuncios/actions.ts:120	Cota de IA e limite de vendas do plano contados em linhas que o usuário apaga ou edita

3. IDOR

Id	Severidade	Arquivo:linha	Descrição
A2	Média	src/app/(app)/produtos/actions.ts:494-513 src/app/(app)/produtos/actions.ts:396-397 (criarVariacao, para comparação)	registrarFoto grava foto em qualquer produto_id e com qualquer path

4. Chaves expostas

Id	Severidade	Arquivo:linha	Descrição
A6	Baixa	scripts/run-migrations.mjs:32-38 scripts/confirm-test-user.mjs:25-31	Scripts conectam como postgres sem verificar o certificado TLS
A8	Informativa	src/app/api/cron/cobrancas/route.ts:8	CRON_SECRET comparado sem tempo constante
A9	Informativa	src/lib/supabase/admin.ts:1-10	Cliente service role sem a guarda server-only

5. Inputs sem tratamento (XSS)

Id	Severidade	Arquivo:linha	Descrição
A10	Informativa	next.config.ts:3-8	Sem Content-Security-Policy nem cabeçalhos de segurança

Detalhe dos achados

A1 · Tabelas filhas aceitam produto de outro dono e a loja pública mostra essas linhas Alta

Categoria: Banco sem tranca

[supabase/migrations/0001_fase1_produtos_estoque_caixa.sql:96-97](#)

```
create policy "produto_variacoes_insert_own" on produto_variacoes
for insert with check (owner_id = auth.uid());
```

[supabase/migrations/0001_fase1_produtos_estoque_caixa.sql:122-123](#)

```
create policy "produto_fotos_insert_own" on produto_fotos
for insert with check (owner_id = auth.uid());
```

[supabase/migrations/0026_vitrine_extras.sql:71 e 75 \(vitrine_publica, SECURITY DEFINER, em produção\)](#)

```
from produto_variacoes v where v.produto_id = p.id
...
from produto_fotos f where f.produto_id = p.id
```

[supabase/migrations/0011_limite_de_fotos.sql:6-14 \(limitar_fotos_produto\)](#)

```
select count(*) from produto_fotos
where produto_id = new.produto_id
and variacao_id is not distinct from new.variacao_id
) >= 10 then raise exception 'Limite de 10 fotos atingido.';
```

O que acontece: As políticas de INSERT/UPDATE conferem só o owner_id da linha nova, nunca se o produto_id pertence ao mesmo dono. As FKs são de coluna única e a checagem de FK ignora RLS, então qualquer conta logada grava em produto_variacoes ou produto_fotos uma linha sua apontando para o produto de outra pessoa. A função vitrine_publica roda como dono do banco e junta variações e fotos só por produto_id.

Como explorar: Conta grátis basta. O atacante pega o id do produto no HTML de /loja/<slug> ou no link ?p=<id>, e com a anon key (pública no bundle) e o próprio JWT faz POST em /rest/v1/produto_variacoes com nome e preco_venda escolhidos, ou em /rest/v1/produto_fotos com uma imagem da própria pasta. A variação falsa (ex.: preço R\$ 1,00) e a foto aparecem na loja da vítima, que não vê nada no app porque o RLS esconde linhas de outro dono. Inserindo 10 fotos, o gatilho de limite passa a recusar fotos novas da própria vítima naquele produto.

Condição: A loja da vítima precisa estar no ar (vitrine ativa e plano pago ou teste de 14 dias). Impacto: Fraude de preço e conteúdo ofensivo na loja de terceiros, dano de reputação, bloqueio do envio de fotos.

A2 · registrarFoto grava foto em qualquer produto_id e com qualquer path Média

Categoria: IDOR

[src/app/\(app\)/produtos/actions.ts:494-513](#)

```
export async function registrarFoto(produtoId: string, path: string, ordem: number, variacaoId: string
| null = null) {
  ...
  const { error } = await supabase.from("produto_fotos").insert({
    owner_id: user.id,
```

```
produto_id: produtoId,
variacao_id: variacaoId,
path,
ordem,
});
```

src/app/(app)/produtos/actions.ts:396-397 (criarVariacao, para comparação)

```
const { data: produto } = await supabase.from("produtos").select(...).eq("id",
produtoId).maybeSingle();
if (!produto) return { ok: false, error: "Produto não encontrado." };
```

O que acontece: Server action é rota pública com POST. registrarFoto recebe produtoId e path do cliente e insere sem conferir que o produto é do usuário (criarVariacao confere) nem que o path está na pasta dele. É o caminho pela própria aplicação do problema A1, sem nem precisar da API REST.

Como explorar: Chamar a action com o id de produto de outra loja e um path qualquer. Mesmo efeito do A1 na loja pública e alimenta o A4 (path de outra pessoa).

Condição: Usuário logado. **Impacto:** Mesmo do A1 e do A4.

A3 · Cota de IA e limite de vendas do plano contados em linhas que o usuário apaga ou edita

Alta

Categoria: Permissão definida no navegador

supabase/migrations/0017_ia_anuncios_e_recibo.sql:29-33

```
create policy "ia_geracoes_update_own" on ia_geracoes
for update using (owner_id = auth.uid());
-- apagar só para devolver a cota quando a geração falha
create policy "ia_geracoes_delete_own" on ia_geracoes
for delete using (owner_id = auth.uid());
```

função ia_do_mes (produção), usada pelo gatilho aplicar_limites_ia

```
select count(*)::integer from ia_geracoes
where owner_id = p_owner
and created_at >= (date_trunc('month', ...))
```

supabase/migrations/0003_clientes_vendas_parcelas.sql:63-64 + vendas_do_mes

```
create policy "vendas_update_own" on vendas
for update using (owner_id = auth.uid()) with check (owner_id = auth.uid());
-- conferido: authenticated tem UPDATE na coluna created_at e DELETE na tabela
```

src/app/(app)/anuncios/actions.ts:120

```
await supabase.from("ia_geracoes").delete().eq("id", registro.id);
```

O que acontece: O limite mensal de IA (10 no grátis, 30 no pago) é count(*) em ia_geracoes, e o dono tem DELETE e UPDATE (inclusive em created_at) nessa tabela. O limite de 30 vendas por mês do plano grátis é count(*) em vendas por created_at, coluna que o dono também pode alterar.

Como explorar: Depois de cada geração, DELETE em /rest/v1/ia_geracoes?id=eq.<id> (ou PATCH created_at para o mês passado) devolve a cota. Em script, gera anúncios sem teto, cada um uma chamada paga à API da Anthropic na conta do UseMarcon. No grátis, PATCH em vendas.created_at libera vendas acima de 30 no mês.

Condição: Usuário logado; para a IA, ANTHROPIC_API_KEY configurada no servidor. **Impacto:** Custo direto e sem teto na API da Anthropic; o limite do plano grátis deixa de valer, reduzindo o motivo para assinar.

A4 · A chave de serviço assina arquivos do Storage cujo caminho o usuário escolhe Média

Categoria: Banco sem tranca

`src/lib/vitrine-servidor.ts:26-28, 31-32 e 36-38`

```
const paths = [...new Set(bruta.produtos.flatMap((p) => p.fotos.map((f) => f.path)))];  
await admin.storage.from("produto-fotos").createSignedUrls(paths, 3600);  
await admin.storage.from("logo-empresa").createSignedUrl(bruta.loja.logo_path, 3600);  
await admin.storage.from("vitrine-banner").createSignedUrls(bannerBruto.paths, 3600)
```

`src/app/r/[token]/page.tsx:33-36 e src/app/r/[token]/pdf/route.ts:47`

```
await admin.storage.from("logo-empresa").createSignedUrl(empresa.logo_path, 3600)
```

`src/app/(app)/configuracoes/actions.ts:58-66`

```
export async function atualizarLogoEmpresa(path: string | null) {  
  ...  
  await supabase.auth.updateUser({ data: { empresa_logo_path: path } });  
}
```

`src/app/(app)/vitrine/actions.ts:113 (validação só na action)`

```
if (!path.startsWith(`${user.id}/`) || path.includes("..")) return { ok: false, error: "Imagem inválida." };
```

O que acontece: Os buckets são privados e a política de Storage limita cada usuário à própria pasta. Mas a loja e o recibo assinam, com a service role (que ignora essa política), caminhos vindos de `produto_fotos.path` (sem CHECK), `vitrines.banner_paths` (o CHECK só limita a 3 itens) e `user_metadata.empresa_logo_path` (o próprio usuário edita com `auth.updateUser`, direto do navegador). A checagem de pasta de adicionarImagemBanner é contornada com PATCH direto em `/rest/v1/vitrines`.

Como explorar: Quem conhece o caminho de um arquivo privado de outra conta grava esse caminho na própria loja ou no próprio logo e recebe um link assinado válido por 1 hora, renovável. Os nomes têm `Date.now()` em milissegundos, então não dá para adivinhar no escuro, mas os caminhos aparecem nas URLs assinadas sempre que a imagem foi pública. Ex.: o vendedor tira um produto da vitrine e as fotos continuam acessíveis para quem guardou o caminho.

Condição: Conhecer o caminho exato do arquivo; para a vitrine, loja própria ativa (o recibo não exige plano). **Impacto:** Leitura de imagens privadas de outras contas e 'despublicar' que não revoga o acesso.

A5 · Métricas da vitrine podem ser infladas por qualquer pessoa, sem limite Baixa

Categoria: Banco sem tranca

`supabase/migrations/0028_vitrine_cupons_metricas.sql:87-117`

```
create or replace function registrar_evento_vitrine(p_slug text, p_tipo text, p_produto uuid default null) ... security definer  
...  
grant execute on function registrar_evento_vitrine(text, text, uuid) to anon, authenticated;
```

src/app/loja/[slug]/actions.ts:27-35

```
await supabase.rpc("registrar_evento_vitrine", { p_slug: s, p_tipo: tipo, p_produto: produtoId ?? null });
```

O que acontece: A função é pública por desenho (a loja abre sem login), mas não há limite por IP, sessão ou janela de tempo.

Como explorar: Um laço chamando /rest/v1/rpc/registrar_evento_vitrine com o slug de qualquer loja soma visitas, pedidos e aberturas de produto à vontade.

Condição: Nenhuma: anon key pública. **Impacto:** Números de visitas e pedidos na página Vitrine deixam de ser confiáveis; crescimento de tabela.

A6 · Scripts conectam como postgres sem verificar o certificado TLS Baixa

Categoria: Chaves expostas

scripts/run-migrations.mjs:32-38

```
user: "postgres",
password,
database: "postgres",
ssl: { rejectUnauthorized: false },
```

scripts/confirm-test-user.mjs:25-31

```
user: "postgres",
password: env.SUPABASE_DB_PASSWORD,
ssl: { rejectUnauthorized: false },
```

O que acontece: A senha do superusuário vem do .env.local (correto, nada hardcoded), mas a conexão aceita qualquer certificado.

Como explorar: Quem estiver no meio da rede (Wi-Fi público, proxy) intercepta a conexão e captura a senha do postgres.

Condição: Rodar os scripts numa rede hostil. **Impacto:** Acesso total ao banco de produção.

A7 · 13 funções SECURITY DEFINER executáveis por anon (correção já em PR) Informativa

Categoria: Banco sem tranca

produção: proacl de ajustar_estoque, cancelar_venda, excluir_produto, pagar_parcela, registrar_entrada/saida_estoque, registrar_venda, uso_do_plano, aplicar_limites_*

```
{=X/postgres,...,anon=X/postgres,authenticated=X/postgres,...}
```

supabase/migrations/0029_revogar_execute_anon.sql

```
revoke execute on function ... from public, anon;
```

O que acontece: Lidas uma a uma: todas barram uid nulo, explicitamente ou porque filtram owner_id = auth.uid(). Não há vazamento hoje; é defesa em profundidade.

Como explorar: Nenhuma conhecida. A migration 0029 (PR HenriqueFauri/Marcon#61) revoga, aguardando aplicação em produção.

Condição: - Impacto: Superfície de ataque maior que o necessário.

A8 · CRON_SECRET comparado sem tempo constante Informativa

Categoria: Chaves expostas

src/app/api/cron/cobrancas/route.ts:8

```
if (!segredo || request.headers.get("authorization") !== `Bearer ${segredo}`) {
```

O que acontece: O webhook do Asaas usa timingSafeEqual; o cron usa !==.

Como explorar: Ataque de tempo pela internet é impraticável aqui; registro por consistência.

Condição: - Impacto: Mínimo.

A9 · Cliente service role sem a guarda server-only Informativa

Categoria: Chaves expostas

src/lib/supabase/admin.ts:1-10

```
import { createClient } from "@supabase/supabase-js";  
// ... Nunca importar em código de cliente.  
export function createAdminClient() {
```

O que acontece: Conferido: nenhum componente 'use client' importa o módulo hoje, e a chave não tem prefixo NEXT_PUBLIC_, então não iria para o bundle. A regra está só num comentário.

Como explorar: Nenhuma hoje.

Condição: - Impacto: Proteção contra regressão futura.

A10 · Sem Content-Security-Policy nem cabeçalhos de segurança Informativa

Categoria: Inputs sem tratamento (XSS)

next.config.ts:3-8

```
const nextConfig: NextConfig = {  
  async redirects() { ... },  
};
```

O que acontece: Nenhum XSS foi encontrado, mas não há CSP, X-Frame-Options/frame-ancestors nem Referrer-Policy que limitem o dano se um aparecer.

Como explorar: Nenhuma direta.

Condição: - Impacto: Defesa em profundidade e proteção contra clickjacking no app logado.

A11 · Validação de cupom sem limite de tentativas Informativa

Categoria: Banco sem tranca

supabase/migrations/0028_vitrine_cupons_metricas.sql:39 e 56-57

```
create or replace function validar_cupom_vitrine(p_slug text, p_codigo text) returns jsonb
...
grant execute on function validar_cupom_vitrine(text, text) to anon, authenticated;
```

O que acontece: Qualquer pessoa testa códigos à vontade. Cupons são feitos para circular, mas um cupom 'secreto' (ex.: para um influenciador) pode ser descoberto por força bruta se for curto.

Como explorar: Laço de tentativas em /rest/v1/rpc/validar_cupom_vitrine.

Condição: Cupom curto ou previsível. **Impacto:** Desconto usado por quem não deveria.

Recomendações priorizadas

P1**Amarrar produto_id ao dono no banco**

Políticas WITH CHECK (ou gatilho) que exigem o produto do mesmo owner_id em produto_variacoes, produto_fotos, produto_anuncios e ia_geracoes; filtrar v.owner_id/f.owner_id = p.owner_id em vitrine_publica; checar posse em registrarFoto; limpar linhas cruzadas existentes. (A1, A2)

P1**Tirar do usuário o controle da contagem dos limites**

Remover DELETE/UPDATE de ia_geracoes para authenticated (devolução de cota por função definir ou pela service role) e impedir alteração de vendas.created_at. (A3)

P2**Conferir a pasta antes de assinar**

CHECK de prefixo owner_id/ em produto_fotos.path e vitrines.banner_paths; as funções públicas só devolvem logo_path dentro da pasta do dono; validação de path em atualizarLogoEmpresa e registrarFoto. (A4)

P2**Aplicar a migration 0029**

Revoga execute de anon nas 13 funções SECURITY DEFINER (PR HenriqueFauri/Marcon#61). (A7)

P3**Limite de taxa nos endpoints públicos**

Limite por IP/sessão em registrar_evento_vitrine e validar_cupom_vitrine. (A5, A11)

P4**Endurecimento**

TLS verificado nos scripts, comparação em tempo constante no cron, import 'server-only' no cliente admin, CSP e cabeçalhos de segurança. (A6, A8, A9, A10)

O achado A7 não vira issue: a correção já está no PR HenriqueFauri/Marcon#61 e só falta aplicar a migration 0029 em produção.

Issues para o GitHub

Cada bloco abaixo é o texto completo de uma issue em Markdown. Copie o que está entre as marcas `---` `ISSUE n` `---` e `---` `FIM ISSUE n` `---`: a primeira linha (`# ...`) é o título.

`---` **ISSUE 1** `---`

```
# [Segurança] Outra conta consegue injetar variações e fotos na loja pública de um vendedor

**Labels sugeridas:** `security`, `alta`
**Achados do relatório:** A1, A2

## Problema e por que é explorável
As políticas RLS de `produto_variacoes` e `produto_fotos` conferem só `owner_id = auth.uid()` da linha nova. Não conferem se o `produto_id` pertence ao mesmo dono, e a FK (coluna única) ignora RLS. A função pública `vitruve_publica` (SECURITY DEFINER) junta variações e fotos só por `produto_id`. A server action `registrarFoto` também insere sem conferir posse.

Qualquer conta logada (grátis serve) pega o id de um produto no HTML de `/loja/<slug>`, chama `POST /rest/v1/produto_variacoes` com a anon key e o próprio JWT, e a variação aparece na loja da vítima com nome e preço escolhidos pelo atacante. O mesmo vale para fotos. Com 10 fotos inseridas, o gatilho `limitar_fotos_produto` passa a recusar fotos da própria vítima.

## Evidência
- `supabase/migrations/0001_fase1_produtos_estoque_caixa.sql:96-97` e `:122-123`
  ```sql
 create policy "produto_variacoes_insert_own" on produto_variacoes
 for insert with check (owner_id = auth.uid());
 create policy "produto_fotos_insert_own" on produto_fotos
 for insert with check (owner_id = auth.uid());
 ...
- `supabase/migrations/0026_vitrine_extras.sql:71` e `:75` (vitrine_publica)
  ```sql
  from produto_variacoes v where v.produto_id = p.id
  from produto_fotos f where f.produto_id = p.id
  ...
- `src/app/(app)/produtos/actions.ts:494-513`
  ```ts
 const { error } = await supabase.from("produto_fotos").insert({
 owner_id: user.id, produto_id: produtoId, variacao_id: variacaoId, path, ordem,
 });
 ...
- `supabase/migrations/0011_limite_de_fotos.sql:6-14`: o limite de 10 conta fotos por `produto_id`, de qualquer dono.

Impacto
Fraude de preço e conteúdo ofensivo na loja de terceiros, dano de reputação para o vendedor e para o UseMarcon, bloqueio do envio de fotos.

Sugestão de correção
1. Migration nova trocando os WITH CHECK de INSERT e UPDATE em `produto_variacoes`, `produto_fotos`, `produto_anuncios` e `ia_geracoes` para:
  ```sql
  with check (
    owner_id = auth.uid()
    and exists (select 1 from produtos p where p.id = produto_id and p.owner_id = auth.uid())
  )
  ...
  (ou um gatilho BEFORE INSERT OR UPDATE que compare `owner_id` com o dono do produto, o que também cobre a service role).
2. Em `vitrine_publica`, filtrar `v.owner_id = p.owner_id` e `f.owner_id = p.owner_id`.
3. Em `limitar_fotos_produto` e `aplicar_limite_foto`, contar só fotos com `owner_id = new.owner_id`.
```

```

4. Em `registrarFoto`, buscar o produto com o cliente da sessão antes de inserir (como `criarVariacao` já faz).
5. Limpar o que já existir:
```sql
select 'variacao', v.id from produto_variacoes v join produtos p on p.id = v.produto_id where
v.owner_id <> p.owner_id
union all
select 'foto', f.id from produto_fotos f join produtos p on p.id = f.produto_id where f.owner_id <>
p.owner_id;
```

## Critérios de aceite
- [ ] Com o JWT da conta B, `POST /rest/v1/produto_variacoes` com `produto_id` de um produto da conta A é recusado (erro de RLS).
- [ ] O mesmo para `produto_fotos`, `produto_anuncios` e `ia_geracoes`.
- [ ] `registrarFoto` com produto de outra conta devolve "Produto não encontrado.".
- [ ] `vitrine_publica` não devolve variação nem foto cujo `owner_id` é diferente do dono do produto.
- [ ] A consulta de limpeza devolve zero linhas em produção depois da migration.
- [ ] Cadastro de variação e envio de foto continuam funcionando para o próprio dono.

```

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

```

# [Segurança] Cota de IA e limite de vendas do plano podem ser zerados pelo próprio usuário

**Labels sugeridas:** `security`, `alta`
**Achados do relatório:** A3

## Problema e por que é explorável
O limite mensal de escritas com IA é `count(*)` em `ia_geracoes` (função `ia_do_mes`), e o dono tem políticas de DELETE e UPDATE nessa tabela, além de privilégio de UPDATE em `created_at`. Basta apagar a linha (ou mudar a data) depois de cada geração para a cota voltar. Cada geração é uma chamada paga à API da Anthropic.

O limite de 30 vendas/mês do plano grátis é `count(*)` em `vendas` por `created_at`, coluna que o dono também pode alterar via `PATCH /rest/v1/vendas`.

## Evidência
- `supabase/migrations/0017_ia_anuncios_e_recibo.sql:29-33`
```sql
create policy "ia_geracoes_update_own" on ia_geracoes
 for update using (owner_id = auth.uid());
create policy "ia_geracoes_delete_own" on ia_geracoes
 for delete using (owner_id = auth.uid());
```
- Função `ia_do_mes` (produção): `select count(*)::integer from ia_geracoes where owner_id = p_owner and created_at >= ...`
- `supabase/migrations/0003_clientes_vendas_parcelas.sql:63-64`: `vendas_update_own` sem restrição de coluna; `has_column_privilege('authenticated','public.vendas','created_at','UPDATE')` = true.
- `src/app/(app)/anuncios/actions.ts:120`: o app apaga a linha para devolver a cota quando a IA falha, motivo da política de DELETE.

## Impacto
Custo sem teto na conta da Anthropic do UseMarcon; o limite do plano grátis deixa de valer.

## Sugestão de correção
1. `drop policy "ia_geracoes_delete_own"` e `drop policy "ia_geracoes_update_own"`.
2. Devolver a cota de outro jeito: função `devolver_cota_ia(p_id uuid)` SECURITY DEFINER que só apaga linha do próprio usuário criada há menos de 2 minutos e ainda sem tokens registrados; gravar os tokens com a service role ou por outra função definer.
3. Em `vendas` (e `ia_geracoes`), impedir mudança de `created_at`: gatilho BEFORE UPDATE com `new.created_at := old.created_at`, ou `revoke update (created_at)` de authenticated.

```

4. Avaliar contar o uso mensal numa tabela de contadores só gravável por função definir, para que apagar vendas também não libere cota.

Critérios de aceite

- [] Com o JWT do usuário, `DELETE /rest/v1/ia\_geracoes?id=eq.<id>` não apaga nada (0 linhas).
- [] `PATCH` em `created\_at` de `ia\_geracoes` e de `vendas` não muda o valor.
- [] Quando a chamada à IA falha, a cota continua sendo devolvida (teste com chave inválida).
- [] Depois de 10 gerações no plano grátis, a 11ª é recusada mesmo após tentativas de apagar ou editar linhas.

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] Chave de serviço assina arquivos do Storage fora da pasta do dono

****Labels sugeridas:**** `security`, `média`
****Achados do relatório:**** A4

Problema e por que é explorável

A loja (`buscarVitrine`) e o recibo (`/r/[token]` e o PDF) assinam com a service role caminhos vindos de `produto\_fotos.path`, `vitrines.banner\_paths` e `user\_metadata.empresa\_logo\_path`. Os três são editáveis pelo usuário (os metadados pelo próprio `auth.updateUser` no navegador), e nada confere que o caminho começa com o id do dono. A checagem de pasta em `adicionarImagemBanner` existe só na server action e é contornada com `PATCH /rest/v1/vitrines`.

Quem conhece o caminho de um arquivo privado de outra conta (por exemplo, de uma foto que já foi pública e saiu da vitrine) consegue um link assinado novo sempre que quiser.

Evidência

- `src/lib/vitrine-servidor.ts:26-28, 31-32, 36-38`

```

```ts
await admin.storage.from("produto-fotos").createSignedUrls(paths, 3600);
await admin.storage.from("logo-empresa").createSignedUrl(bruta.loja.logo_path, 3600);
await admin.storage.from("vitrine-banner").createSignedUrls(bannerBruto.paths, 3600)
```

```
- `src/app/r/[token]/page.tsx:33-36` e `src/app/r/[token]/pdf/route.ts:47`: mesmo padrão com o logo.
- `src/app/(app)/configuracoes/actions.ts:66`: `updateUser({ data: { empresa\_logo\_path: path } })` sem validar o path.
- `src/app/(app)/vitrine/actions.ts:113`: `` path.startsWith(`\${user.id}/`) `` só na action. No banco, `vitrines\_banner\_paths\_check` só limita a 3 itens.

Impacto

Leitura de imagens privadas de outras contas; tirar um produto da vitrine não revoga o acesso a quem guardou o caminho.

Sugestão de correção

1. CHECK em `produto\_fotos`: `path like owner\_id::text || '/'` and path not like '%.%'`.
2. CHECK (ou gatilho) em `vitrines`: todo item de `banner\_paths` começa com `owner\_id::text || '/'`.
3. Em `vitrine\_publica` e `recibo\_publico`, devolver `logo\_path` só se `starts\_with(logo\_path, owner\_id::text || '/')`. Melhor ainda: mover o logo de `user\_metadata` para uma coluna de tabela com CHECK.
4. Validar o prefixo também em `atualizarLogoEmpresa` e `registrarFoto`.
5. Rodar uma consulta para achar caminhos atuais fora da pasta do dono.

Critérios de aceite

- [] Inserir em `produto\_fotos` um path de outra pasta é recusado pelo banco.
- [] `PATCH /rest/v1/vitrines` com `banner\_paths` de outra pasta é recusado.
- [] Logo apontado (via `auth.updateUser`) para arquivo de outra conta não aparece no recibo nem na loja.
- [] Envio de fotos, banner e logo continua funcionando para o dono.

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

```
# [Segurança] Métricas da vitrine e validação de cupom sem limite de taxa

**Labels sugeridas:** `security`, `baixa`
**Achados do relatório:** A5, A11

## Problema e por que é explorável
`registrar_evento_vitrine` e `validar_cupom_vitrine` são públicas por desenho, mas não têm limite de chamadas. Qualquer pessoa infla visitas, pedidos e aberturas de produto de qualquer loja, e testa códigos de cupom por força bruta.

## Evidência
- `supabase/migrations/0028_vitrine_cupons_metrics.sql:87-117` (registrar_evento_vitrine, grant para anon na linha 117)
- `supabase/migrations/0028_vitrine_cupons_metrics.sql:39` e `:56-57` (validar_cupom_vitrine, grant para anon)
- `src/app/loja/[slug]/actions.ts:11-35`: as actions só validam formato.

## Impacto
Métricas não confiáveis na página Vitrine; cupom restrito descoberto e usado por terceiros.

## Sugestão de correção
1. Revogar execute de anon nas duas funções e chamá-las só pelas server actions (que rodam no servidor).
2. Nas actions, limitar por IP (cabeçalho `x-forwarded-for` da Vercel) e por slug numa janela curta, com tabela de contagem ou KV.
3. Contar visita no máximo uma vez por visitante por dia (cookie anônimo).
4. Orientar no app que cupom restrito tenha 8 caracteres ou mais.

## Critérios de aceite
- [ ] Chamada direta a `/rest/v1/rpc/registrar_evento_vitrine` com a anon key é recusada.
- [ ] 100 chamadas seguidas da mesma origem à action contam no máximo o limite definido.
- [ ] Validação de cupom bloqueia temporariamente após N tentativas erradas por IP.
```

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

```
# [Segurança] Endurecimento: TLS nos scripts, comparação do CRON_SECRET, server-only e CSP

**Labels sugeridas:** `security`, `baixa`
**Achados do relatório:** A6, A8, A9, A10

## Problema e por que é explorável
Pequenos ajustes de defesa em profundidade, agrupados para não gerar várias issues:
- Os scripts de banco conectam como `postgres` com `ssl: { rejectUnauthorized: false }`: numa rede hostil a senha do superusuário pode ser capturada.
- O cron compara o segredo com `!==` (o webhook já usa `timingSafeEqual`).
- O cliente service role não importa `server-only`; a regra está só num comentário.
- Não há CSP nem cabeçalhos como `frame-ancestors`, `X-Frame-Options` e `Referrer-Policy`.

## Evidência
- `scripts/run-migrations.mjs:38` e `scripts/confirm-test-user.mjs:31`: `ssl: { rejectUnauthorized: false }`
- `src/app/api/cron/cobrancas/route.ts:8`: `` request.headers.get("authorization") !== `Bearer ${segredo}` ``
- `src/lib/supabase/admin.ts:1-10`: sem `import "server-only"`
- `next.config.ts:3-8`: só `redirects()`, sem `headers()`

## Impacto
Baixo hoje; reduz o dano de erros futuros e o risco de vazar a senha do banco.

## Sugestão de correção
```

1. Nos scripts, usar ``ssl: { rejectUnauthorized: true, ca: <certificado do Supabase> }`` ou a connection string do pooler com ``sslmode=verify-full``.
2. No cron, comparar com ``timingSafeEqual``, igual ao webhook.
3. Adicionar o pacote ``server-only`` e ``import "server-only"`` em ``src/lib/supabase/admin.ts``, ``src/lib/asaas.ts`` e ``src/lib/ia-anuncio.ts``.
4. Em ``next.config.ts``, ``headers()`` com ``Content-Security-Policy`` (scripts próprios, GA, Clarity; ``frame-ancestors 'none'``), ``Referrer-Policy: strict-origin-when-cross-origin`` e ``X-Content-Type-Options: nosniff``. Começar em ``Content-Security-Policy-Report-Only``.

Critérios de aceite

- [] Scripts falham ao conectar se o certificado não for o do Supabase.
- [] Cron usa comparação em tempo constante.
- [] Importar ``@/lib/supabase/admin`` num componente ``'use client'`` quebra o build.
- [] Resposta das páginas traz CSP (report-only no primeiro deploy) sem quebrar GA, Clarity e a loja.

--- FIM ISSUE 5 ---